COMPUTER VISION

LAB SHEET :04

```
# Name: Deepti
# CUID:24250168
# Roll No.: 16   Sec:B

import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. LOAD THE IMAGE
# Keep image.jpg in the same folder as this Python file
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

if img is None:
    print("Error: Image could not be loaded.")
    print("Make sure image.jpg is in the same folder as this Python file.")
    exit()
else:
    print("Image loaded successfully.")
    print("Image size:", img.shape)

# 2. DISPLAY ORIGINAL IMAGE
plt.figure(figsize=(6, 5))
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

```
# 3. COMPUTE DISCRETE FOURIER TRANSFORM (DFT)
# Convert image to frequency domain
dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)

# Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

# 4. DISPLAY MAGNITUDE SPECTRUM
# Calculate magnitude spectrum
magnitude_spectrum = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)

# Apply logarithmic scaling for better visualization
magnitude_spectrum = 20 * np.log(
    magnitude_spectrum + 1
)

plt.figure(figsize=(6, 5))
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()

# 5. CREATE LOW-PASS FREQUENCY FILTER
rows, cols = img.shape
```

```
crow, ccol = rows // 2, cols // 2

# Create a mask filled with zeros
low_pass_mask = np.zeros((rows, cols, 2), np.float32)

# Define radius of low-frequency region
radius = 50

# Set center region to 1
for y in range(rows):
    for x in range(cols):
        distance = np.sqrt(
            (x - ccol) ** 2 +
            (y - crow) ** 2
        )

        if distance <= radius:
            low_pass_mask[y, x] = 1

# 6. APPLY LOW-PASS FILTER
low_pass_dft = dft_shift * low_pass_mask

# Shift frequency components back
low_pass_inverse_shift = np.fft.ifftshift(low_pass_dft)

# Perform inverse DFT
low_pass_image = cv2.idft(low_pass_inverse_shift)
```

```
# Calculate magnitude
low_pass_image = cv2.magnitude(
    low_pass_image[:, :, 0],
    low_pass_image[:, :, 1]
)

# Normalize image
low_pass_image = cv2.normalize(
    low_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# 7. CREATE HIGH-PASS FREQUENCY FILTER
# Create a mask filled with ones
high_pass_mask = np.ones((rows, cols, 2), np.float32)

# Remove the low-frequency center
for y in range(rows):
    for x in range(cols):
        distance = np.sqrt(
            (x - ccol) ** 2 +
            (y - crow) ** 2
        )

        if distance <= radius:
```

```
            high_pass_mask[y, x] = 0

# 8. APPLY HIGH-PASS FILTER
high_pass_dft = dft_shift * high_pass_mask

# Shift frequency components back
high_pass_inverse_shift = np.fft.ifftshift(high_pass_dft)

# Perform inverse DFT
high_pass_image = cv2.idft(high_pass_inverse_shift)

# Calculate magnitude
high_pass_image = cv2.magnitude(
    high_pass_image[:, :, 0],
    high_pass_image[:, :, 1]
)

# Normalize image
high_pass_image = cv2.normalize(
    high_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# 9. DISPLAY LOW-PASS AND HIGH-PASS RESULTS
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()

# 10. SAVE FILTERED IMAGES

cv2.imwrite("low_pass_image.jpg", low_pass_image)
cv2.imwrite("high_pass_image.jpg", high_pass_image)
```

```
cv2.imwrite("magnitude_spectrum.jpg", magnitude_spectrum)

print("Low-pass filtered image saved successfully!")
print("High-pass filtered image saved successfully!")
print("Magnitude spectrum saved successfully!")

# OBSERVATIONS

# Original Image:
# The original grayscale image contains both low-frequency and high-frequency components.

# Frequency Spectrum:
# The Fourier Transform represents the image in terms of its frequency components.
# Low-frequency components are concentrated near the center of the shifted spectrum.

# Low-Pass Filter:
# The low-pass filter removes high-frequency components from the image.
# It produces a smoother image and helps reduce noise and fine details.

# High-Pass Filter:
# The high-pass filter removes low-frequency components.
# It emphasizes edges, fine details, and sudden changes in image intensity.

# Inverse Fourier Transform:
# The inverse Fourier Transform converts the filtered frequency-domain image
# back into the spatial domain.
```

Comparison:

Low-pass filtering produces a smoother image, while high-pass filtering

emphasizes edges and fine image details.

QUESTIONS AND ANSWERS

1. What is the Fourier Transform? Why is it important in digital image processing?

Answer: The Fourier Transform is a mathematical technique used to convert an image

from the spatial domain into the frequency domain.

It represents an image using different frequency components.

It is important because it allows us to analyze and manipulate low-frequency

and high-frequency information for noise removal, enhancement, and filtering.

2. Differentiate between the spatial domain and the frequency domain.

Answer: In the spatial domain, image processing is performed directly on the pixels

of an image and their neighboring pixels.

In the frequency domain, the image is represented using frequency components

after applying a Fourier Transform.

Spatial domain methods use operations such as smoothing, sharpening, and edge detection.

Frequency domain methods modify selected frequency components for filtering and enhancement.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Answer: The Discrete Fourier Transform converts a digital image from the spatial domain

into its frequency representation.

It helps identify different frequency components present in an image.

DFT is useful for designing frequency-domain filters, removing noise,

enhancing features, and analyzing the frequency spectrum.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Answer: The zero-frequency component normally appears at the corners of the frequency spectrum.

Shifting it to the center makes the spectrum easier to visualize and analyze.

After shifting, low-frequency components appear near the center and high-frequency

components appear farther away from the center.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Answer: Low-pass filters allow low-frequency components to pass and suppress high-frequency components.

They are mainly used for image smoothing and noise reduction.

High-pass filters suppress low-frequency components and allow high-frequency components to pass.

They are mainly used for edge enhancement and highlighting fine image details.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

Answer: The Inverse Fourier Transform converts the filtered frequency-domain image

back into the spatial domain.

It is used after frequency filtering to reconstruct the final filtered image.

This allows the processed image to be displayed as a normal spatial-domain image.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Answer: Frequency-domain filtering provides direct control over different frequency components.

It allows low-frequency and high-frequency information to be selectively removed or enhanced.

This makes it useful for noise reduction, smoothing, sharpening, and edge enhancement.

It is especially useful when selective frequency manipulation is required.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Answer: Fourier Transform is used in medical image enhancement, satellite image analysis,

image restoration, and biometric systems.

It helps analyze frequency components and improve image quality in these applications.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Answer: Spatial-domain filtering directly processes image pixels using kernels

and is simple to implement.

Frequency-domain filtering transforms the image into the frequency domain

and modifies its frequency components.

Frequency-domain filtering can be efficient for certain large filtering operations,

while spatial filtering is convenient for local image processing.

Both techniques are widely used for image enhancement and computer vision applications.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Answer: Frequency-domain filtering can suppress unwanted frequency components such as noise

while preserving useful image information.

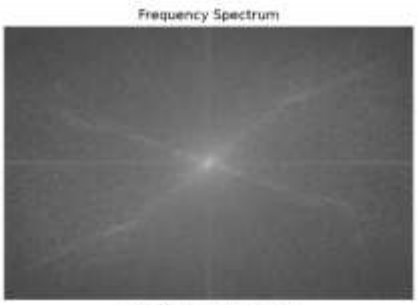
It can also enhance high-frequency components containing edges and fine details.

This improves the quality of restored images and makes important features more visible.

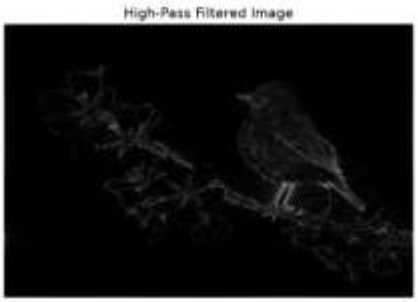
Therefore, it can improve subsequent feature extraction and computer vision tasks.

OUTPUT


Original Image


Frequency Spectrum


Low-Pass Filtered Image


High-Pass Filtered Image